

PROJET 5

GESTION DES RESSOURCES EN PÉTROLE

Elaboré par l'équipe 15:

BOURBIA OTHMEN
GUIZAINI CHAIMA
HAOUAMI WISSAL

JELITI GHOFRANE
KHADHRAOUI ELEE
MIDOUNI KHALIL

Plan

1. Présentation du problème
2. Modèle mathématique
3. Exécution Python
4. Solutions Heuristiques
5. Exécution Python



Présentation du problème

une entreprise doit poser des pipelines pour transporter le brut vers des raffineries potentielles après avoir découvert de nouveaux gisements de pétrole.

- Le problème consiste à **déterminer le type de pipelines à installer** entre les différentes zones afin de transporter **simultanément** toutes les quantités de demandes de pétrole brut

? Comment minimiser les coûts fixes d'installation tout en prenant en considération les contraintes spécifiques imposées

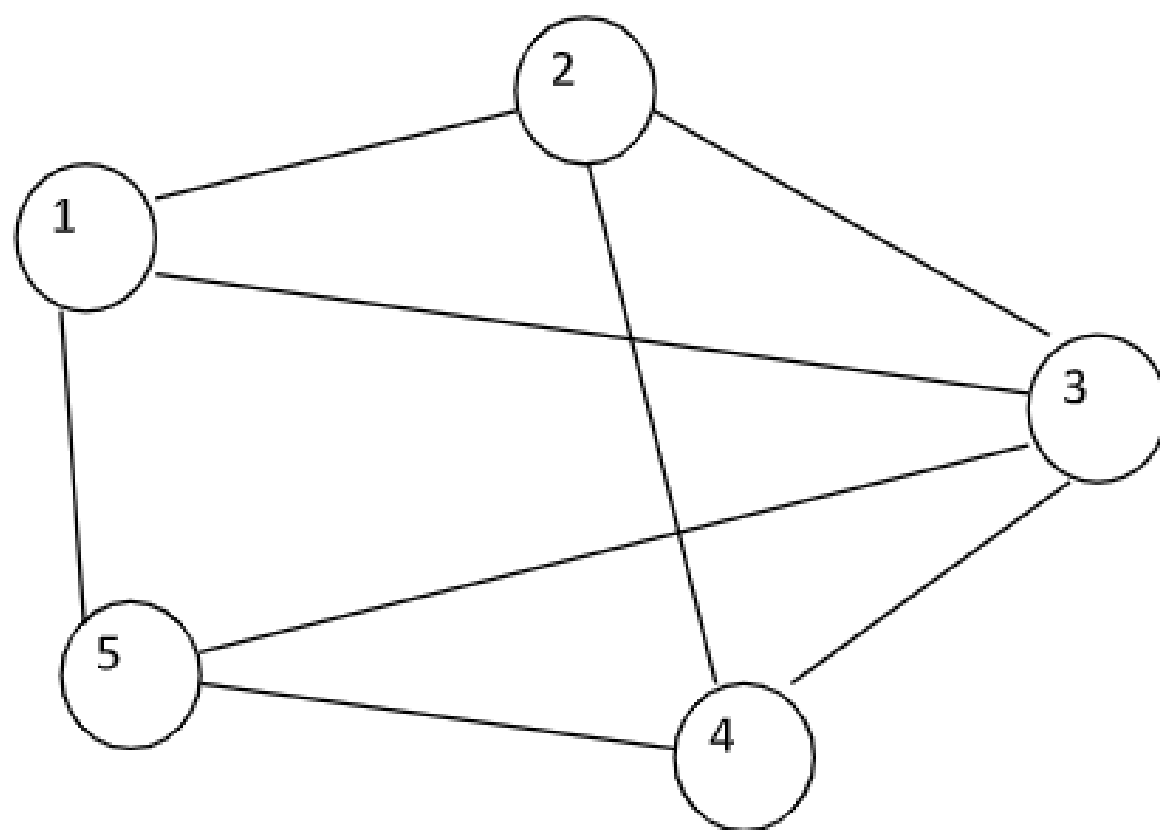


Figure 1 : Zone de pétrole

- Nombre de zones : 5
- Nombre de liaisons entre les zones : 16
- Nombre de demandes à transporter : 20

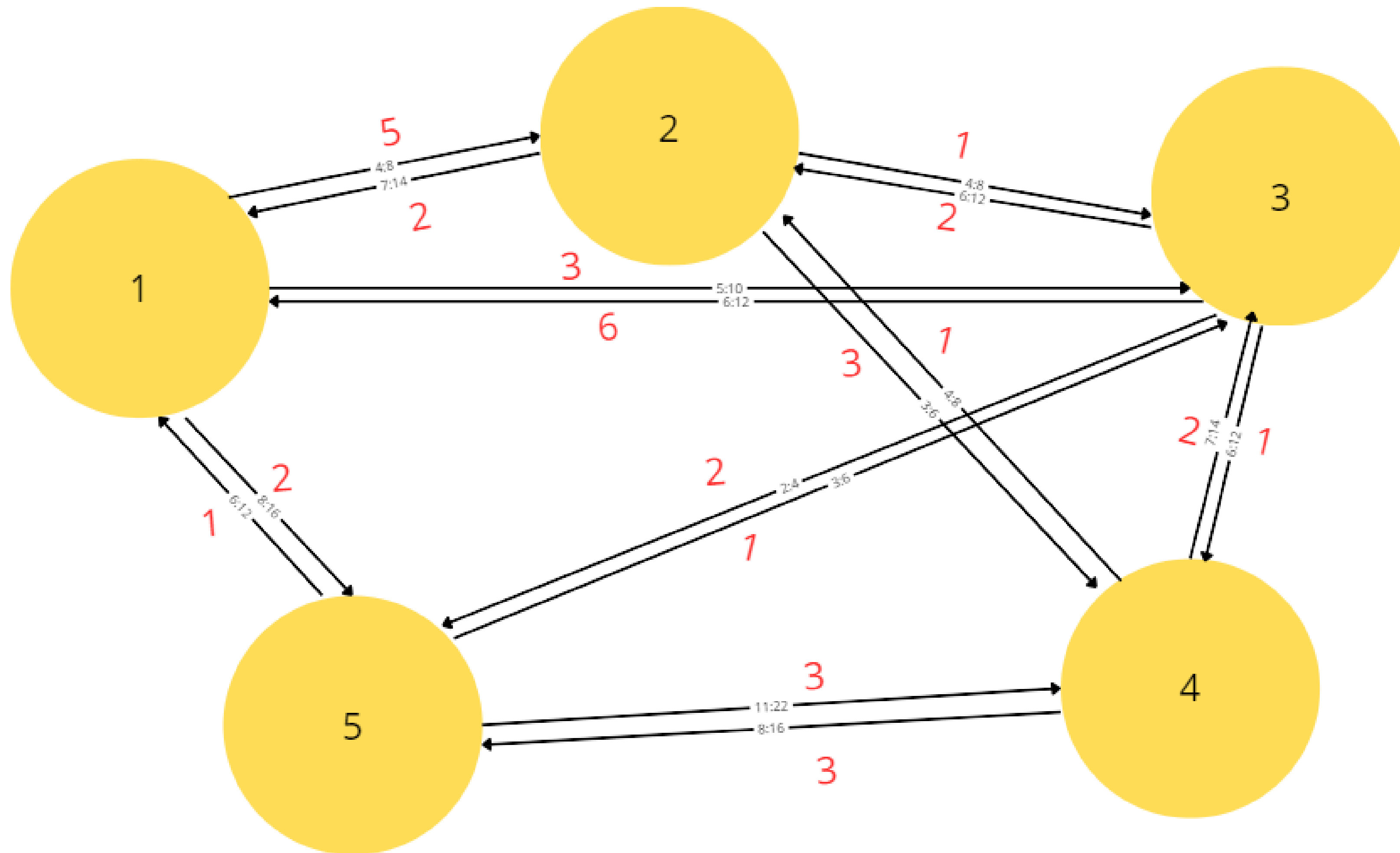
i	j	Nbre de liaisons sur l'arc ij	coût 1	capacité 1	coût 2	capacité 2
1	2	2	4	8	7	14
1	3	2	5	10	6	12
1	5	2	6	12	7	16
2	1	2	4	8	8	14
2	3	2	4	8	6	12
2	4	2	3	6	4	8
3	1	2	5	10	6	12
3	2	2	4	8	6	12
3	4	2	6	12	7	14
3	5	2	2	4	3	6
4	2	2	3	6	4	8
4	3	2	6	12	7	14
4	5	2	8	16	11	22
5	1	2	6	12	8	16
5	3	2	2	4	3	6
5	4	2	8	16	11	22

Contraintes de la demande

Les demandes sont définies entre différentes paires de source-destination.

Source de la demande	Destination de la demande	Quantité de la demande
1	2	5
1	3	3
1	4	7
1	5	2
2	1	2
2	3	1
2	4	3
2	5	4
3	1	6
3	2	2

Source de la demande	Destination de la demande	Quantité de la demande
3	4	1
3	5	2
4	1	4
4	2	1
4	3	2
4	5	3
5	1	1
5	2	2
5	3	1
5	4	3



MODELISATION MATHEMATIQUE

- **Variables de décision**
- **Fonction Objectif**
- **Les sous contraintes**
- **Les contraintes de signe**

Variables de décision

On déclare les variables de décision suivantes, afin de répondre au problème:

f_{ij}^n : variables qui correspondent aux flux qui circulent entre les sites i et j et à la demande n , tels que $i, j \in \{1, \dots, 5\}$ et $n \in \{1, \dots, 20\}$

x_l^e : variables binaires qui indiquent si une pipeline de type l est posée sur l'arrêt e , où $e \in E$ et $l \in \{1, \dots, L_e\}$

E : l'ensemble des arcs du problème

$1, \dots, L_e$: les types de pipeline donnés

Fonction Objectif

Le but est de minimiser le coût d'installations des pipelines entre les différents sites, tout en tenant compte des possibilités de liaison.

$$\min Z = \sum_{e \in E} \sum_{l \in \{1, \dots, L_e\}} c_l^e x_l^e \quad (1)$$

c_l^e étant le coût de pose d'une pipeline de type l sur l'arrêt e .

Les sous contraintes

au plus une seule canalisation peut être posée entre deux zones distinctes

Nombres de pipeline à installer:

$$\sum_{l \in \{1, \dots, L_e\}} x_l^e \leq 1, \forall e \in E \quad (2)$$

On ne peut pas poser plus qu'une pipeline sur une même arrête.

Les sous contraintes

Il est impératif de respecter la capacité de chaque canalisation en ne dépassant pas la quantité de pétrole à transporter dans les deux sens.

Flux et capacité:

$$\sum_{n \in \{1, \dots, 20\}} f_{ij}^n + \sum_{n \in \{1, \dots, 20\}} f_{ji}^n \leq \sum_{l \in \{1, \dots, L_e\}} \text{cap}_l^e x_l^e \quad (3)$$

où cap_l^e est la capacité de la pipeline de type l posée sur l'arrêt e .

La quantité de pétrole circulant dans une pipeline doit être inférieure à la capacité de la pipeline installée.

Les sous contraintes

transporter simultanément toutes les quantités de demandes de pétrole brut

Demande et quantité de pétrole:

$\forall n \in \{1, \dots, 20\}$,

$$\sum_{j:(i,j) \in \{1, \dots, 6\}^2} f_{ij}^n - \sum_{j:(i,j) \in \{1, \dots, 6\}^2} f_{ji}^n = \begin{cases} d_n & \text{si } i \text{ est une source} \\ -d_n & \text{si } i \text{ est une destination} \\ 0 & \text{sinon} \end{cases} \quad (4)$$

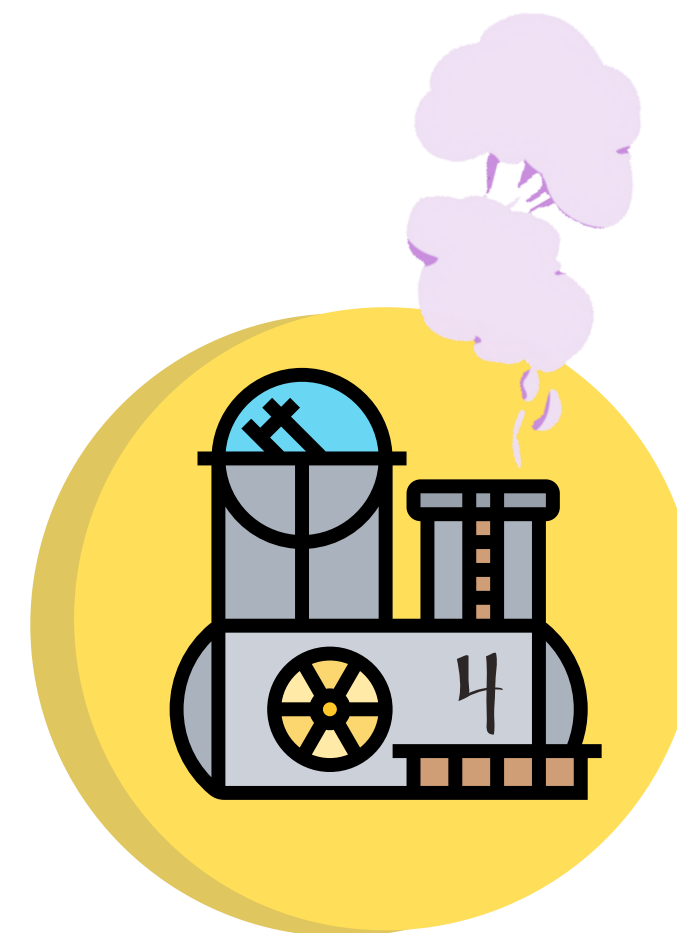
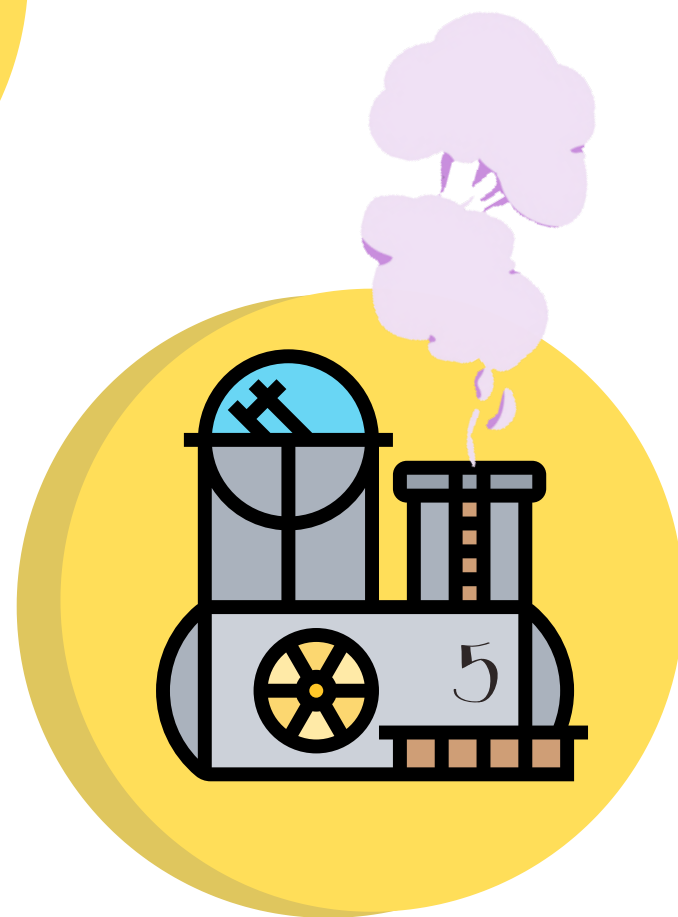
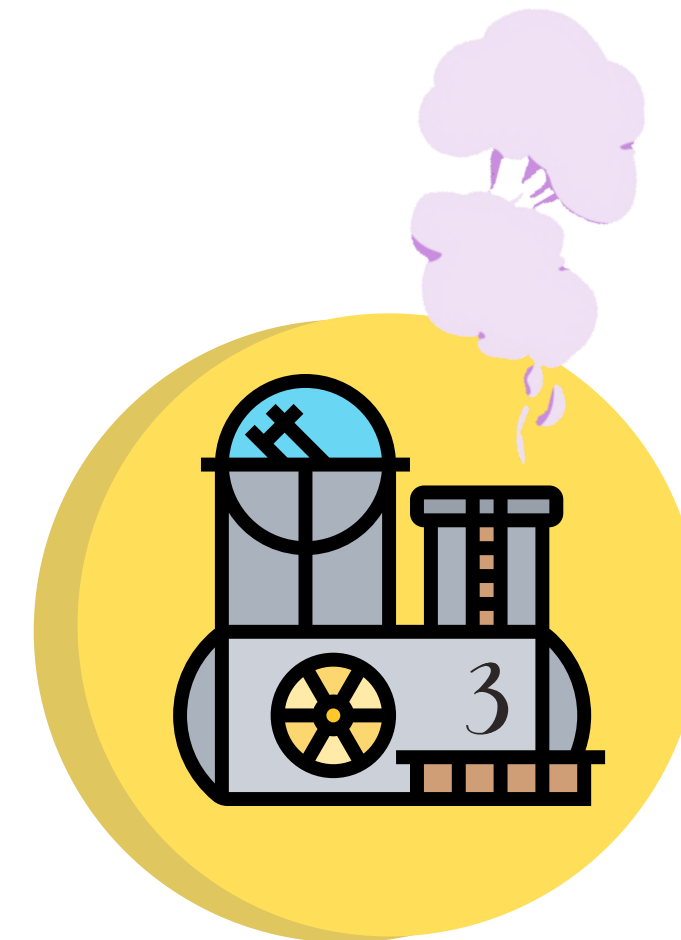
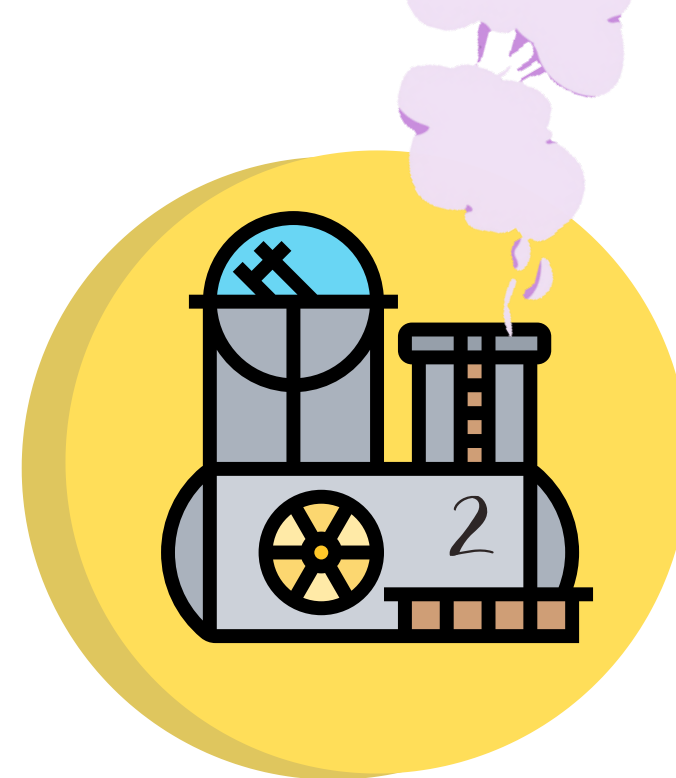
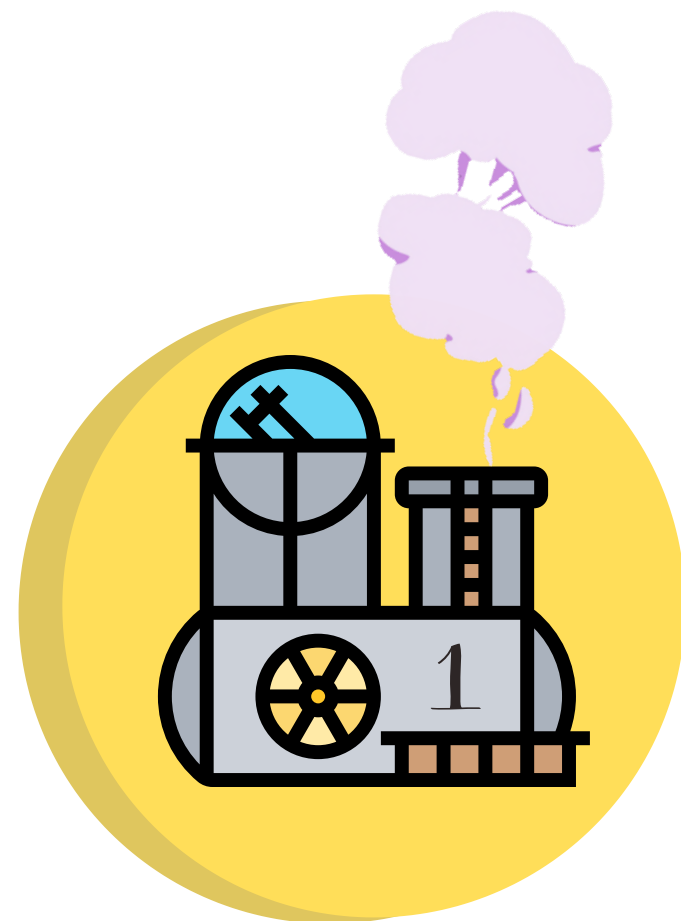
Les restrictions de signe

$$\forall (i, j) \in 1, \dots, 6\}^2, \forall n \in \{1, \dots, 20\}, f_{ij}^n \geq 0 \quad (5)$$

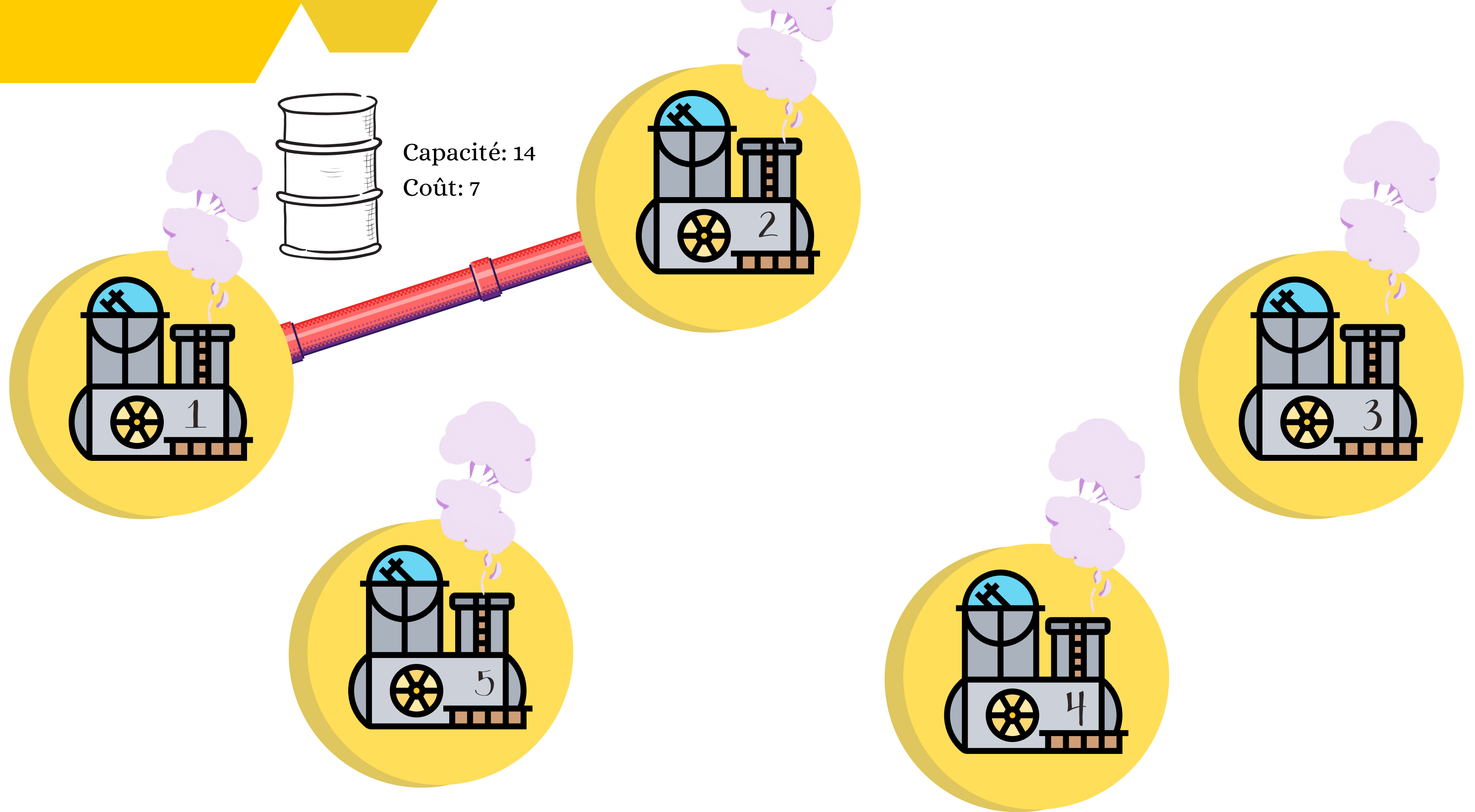
$$\forall e \in E, \forall l \in \{1, \dots, L_e\}, x_l^e \in \{0, 1\} \quad (6)$$

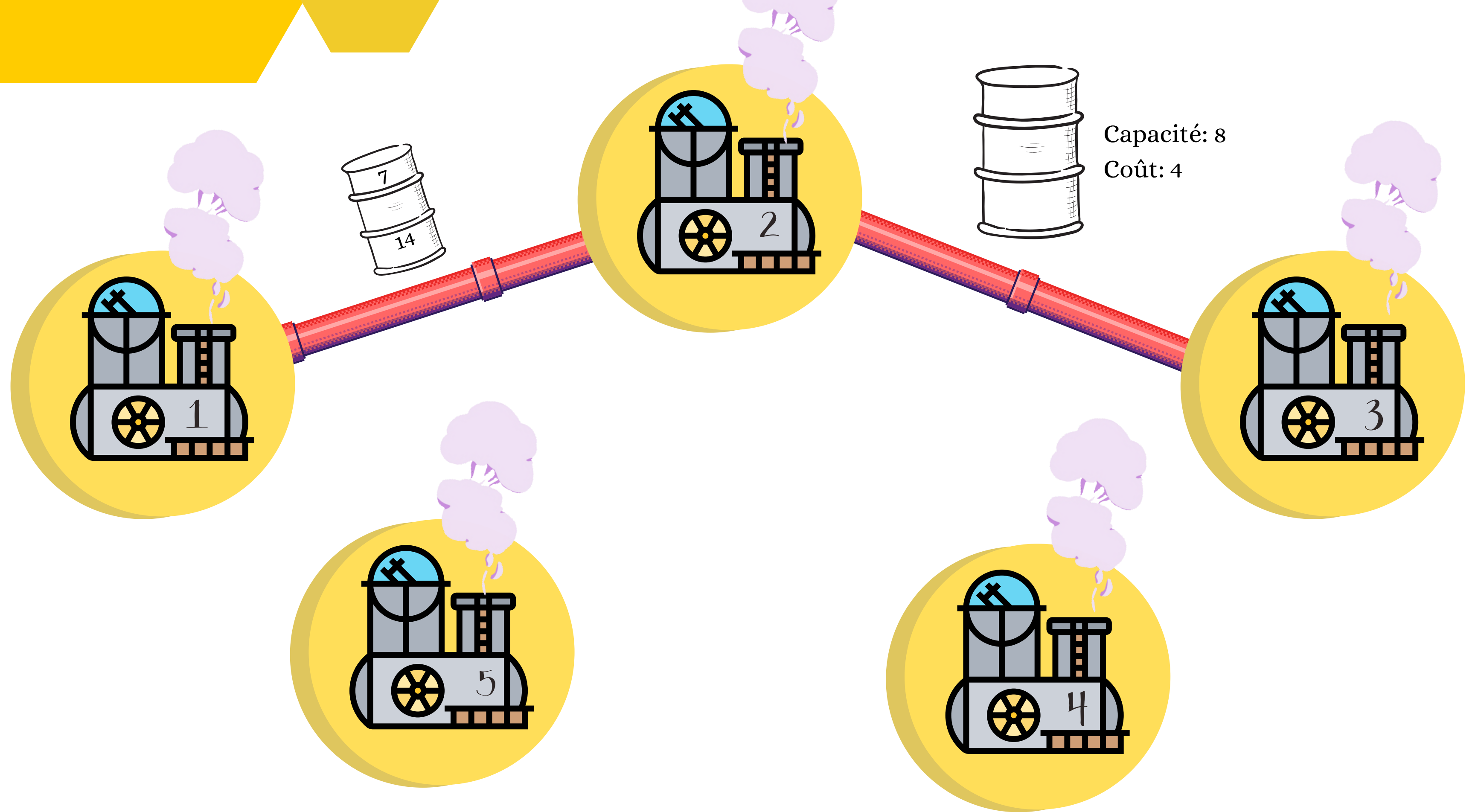
Exécution Python

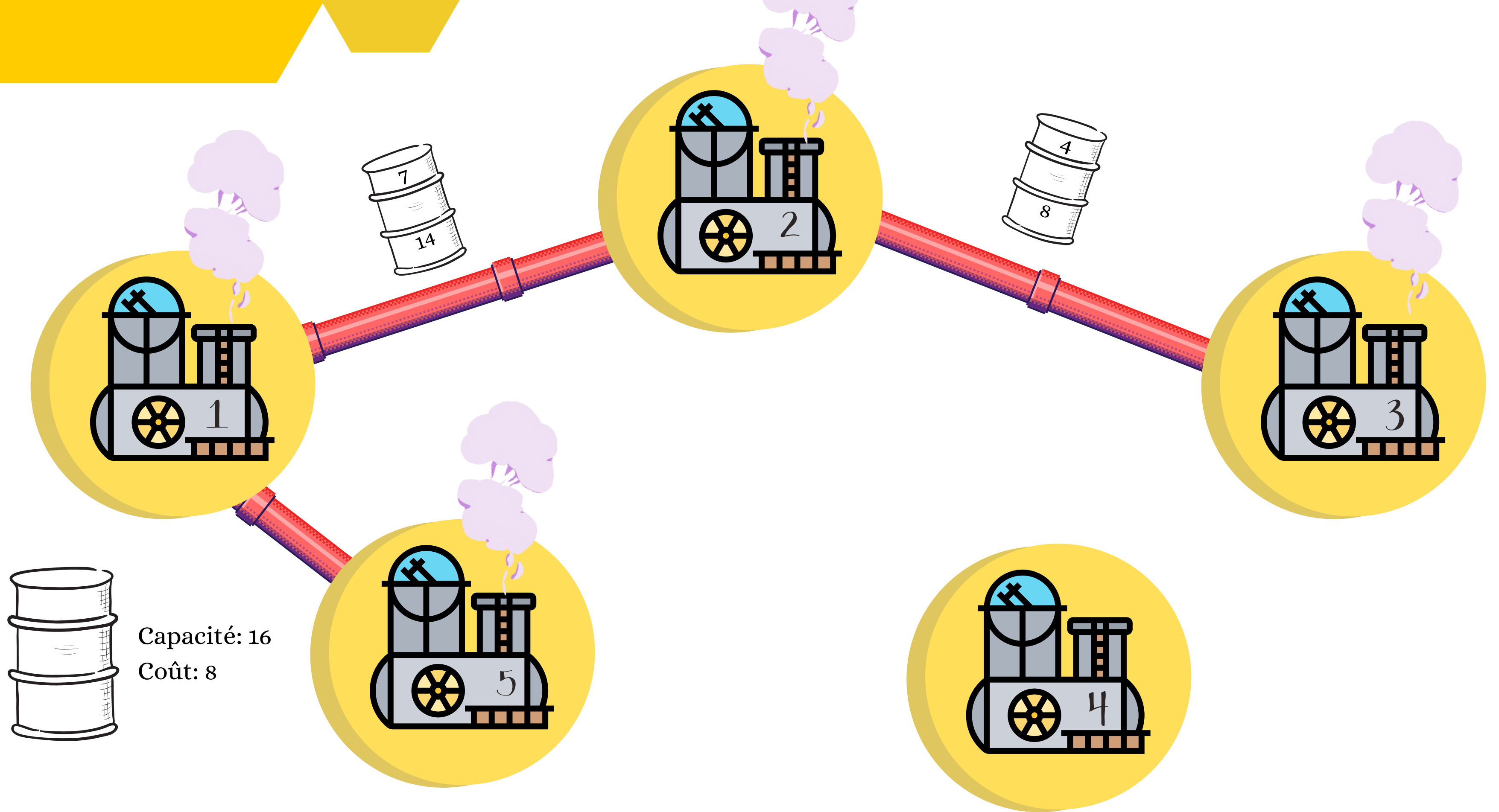
- **Les bibliothèques utilisées**
- **Les données**
- **Variables de décision**
- **Fonction objectif**
- **Les sous contraintes**

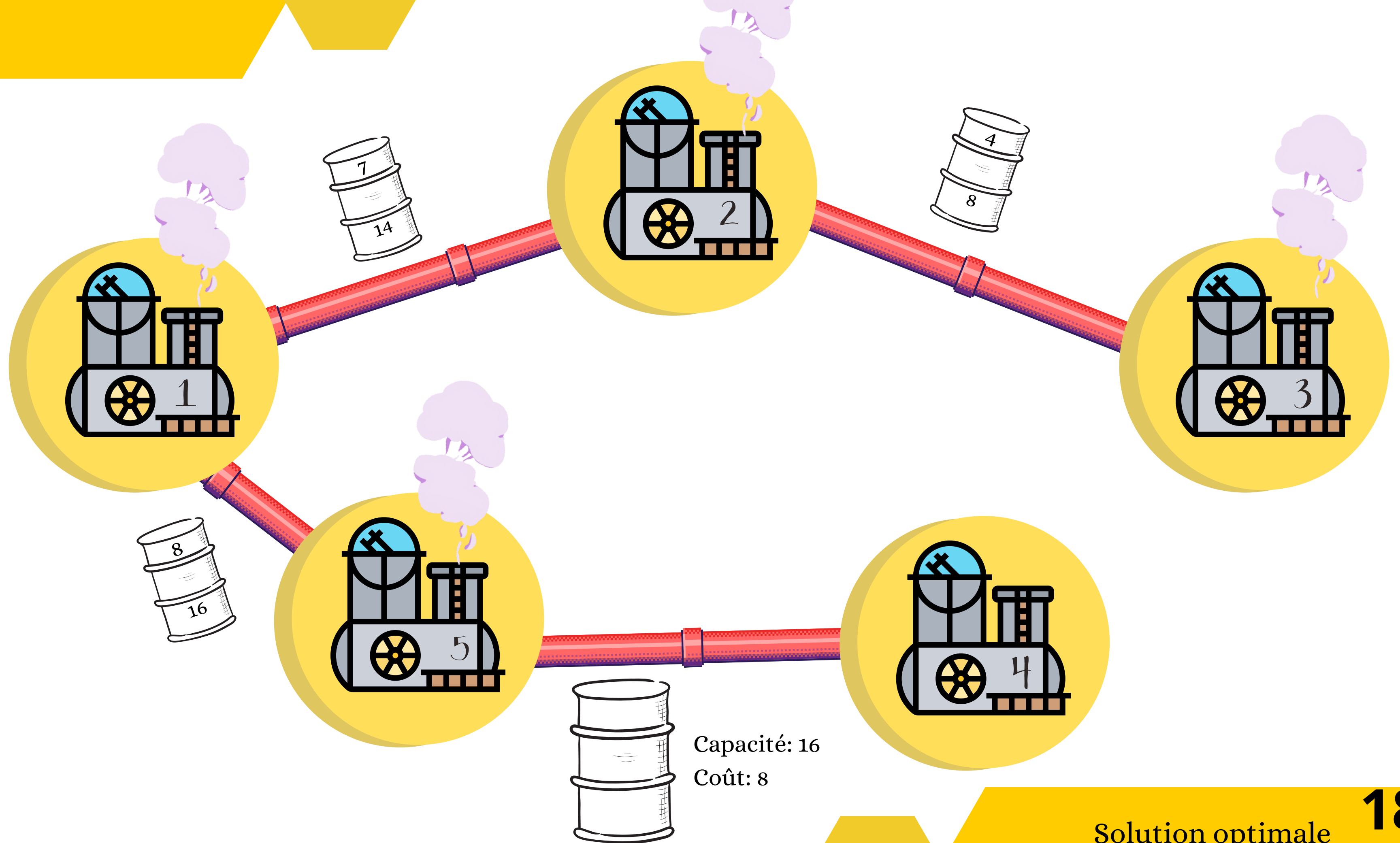


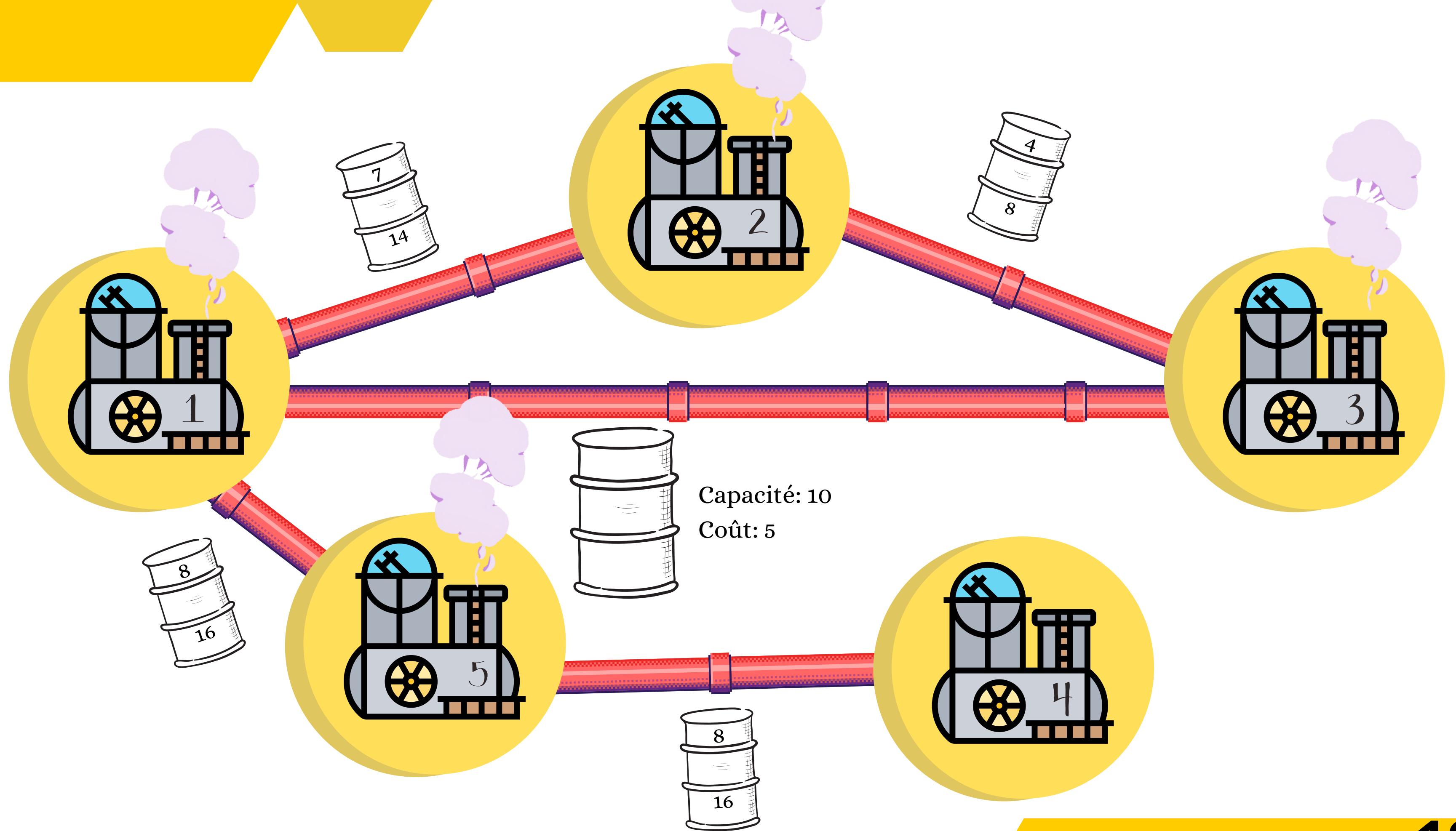
Solution optimale

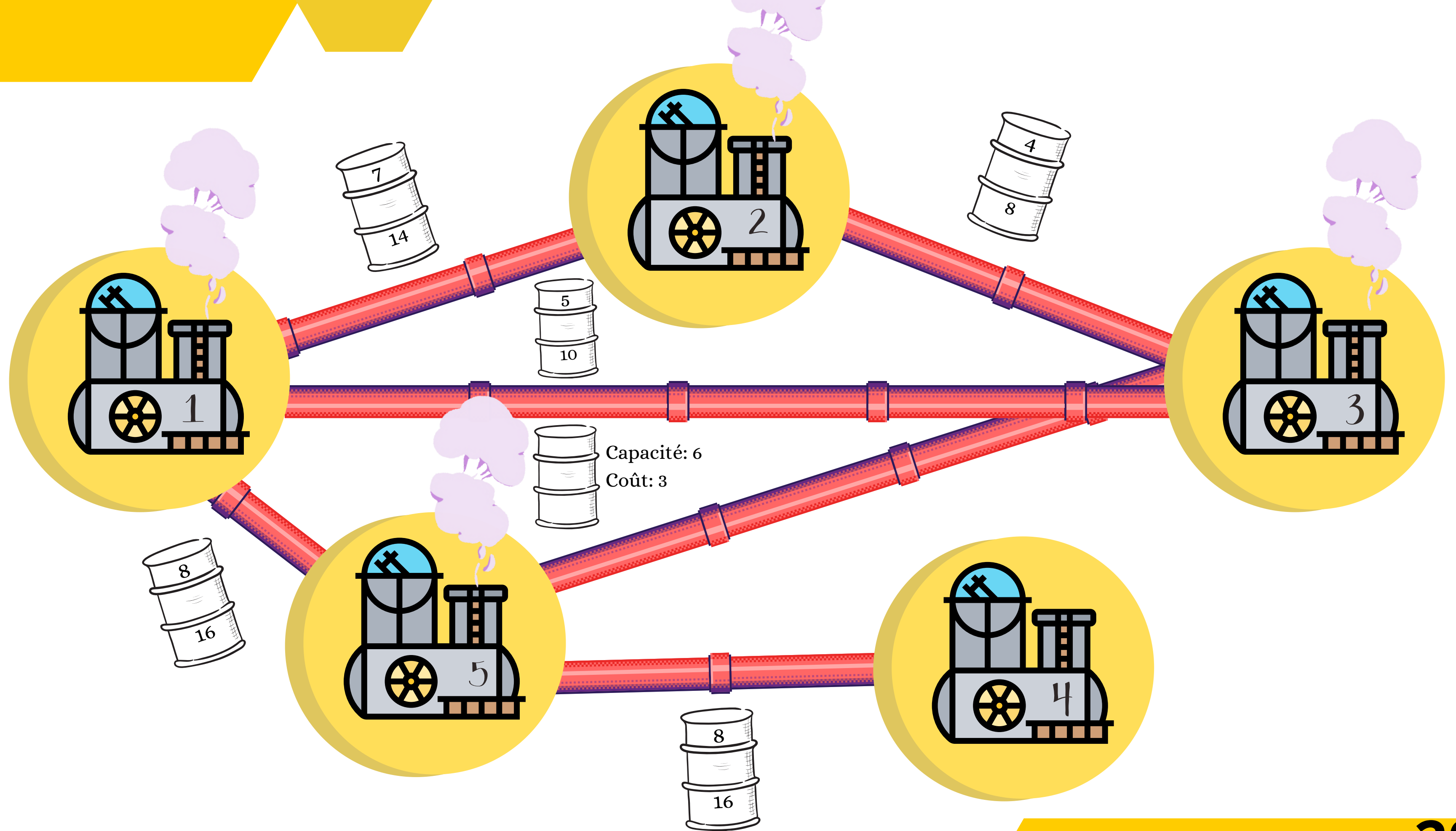


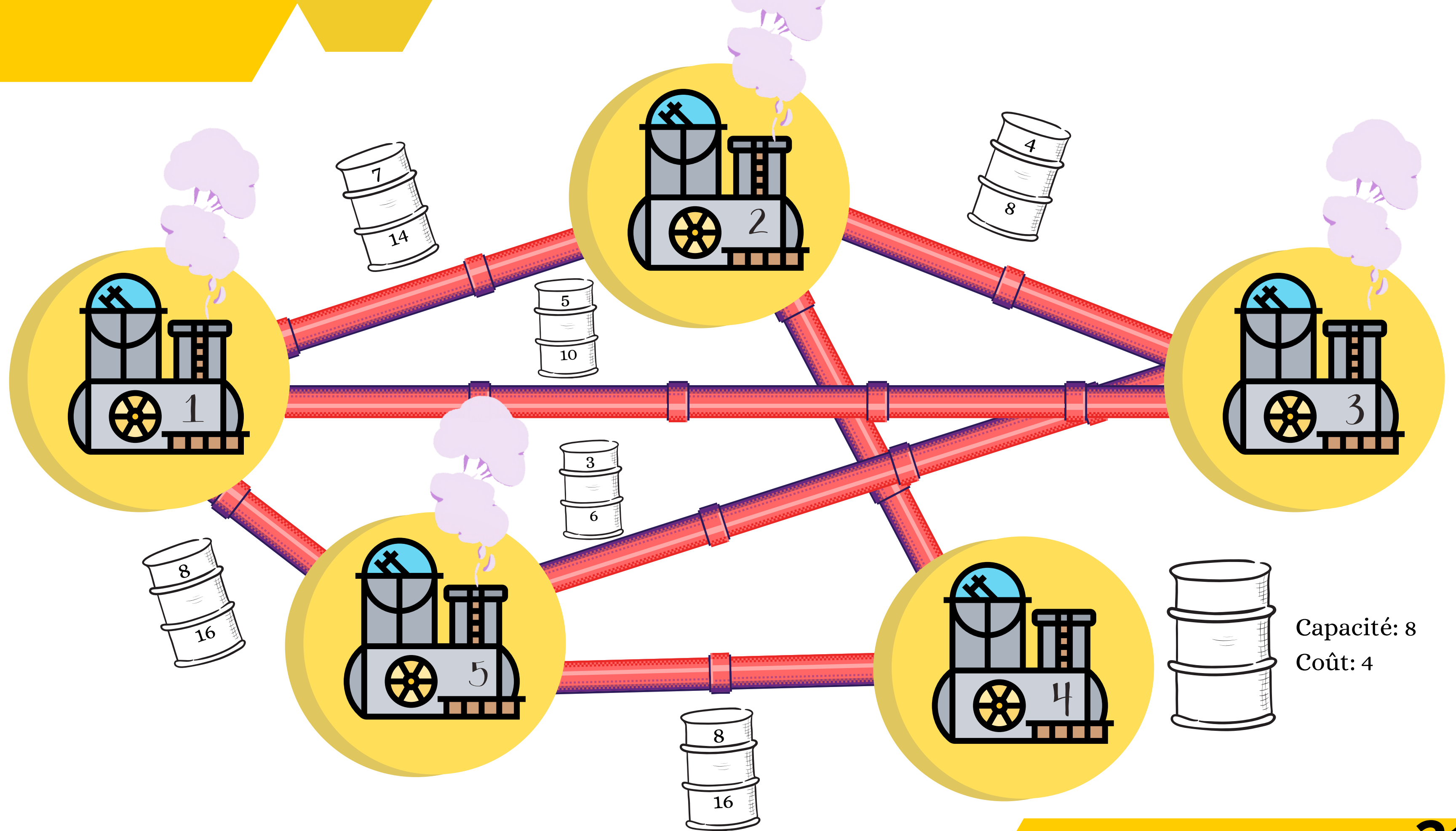


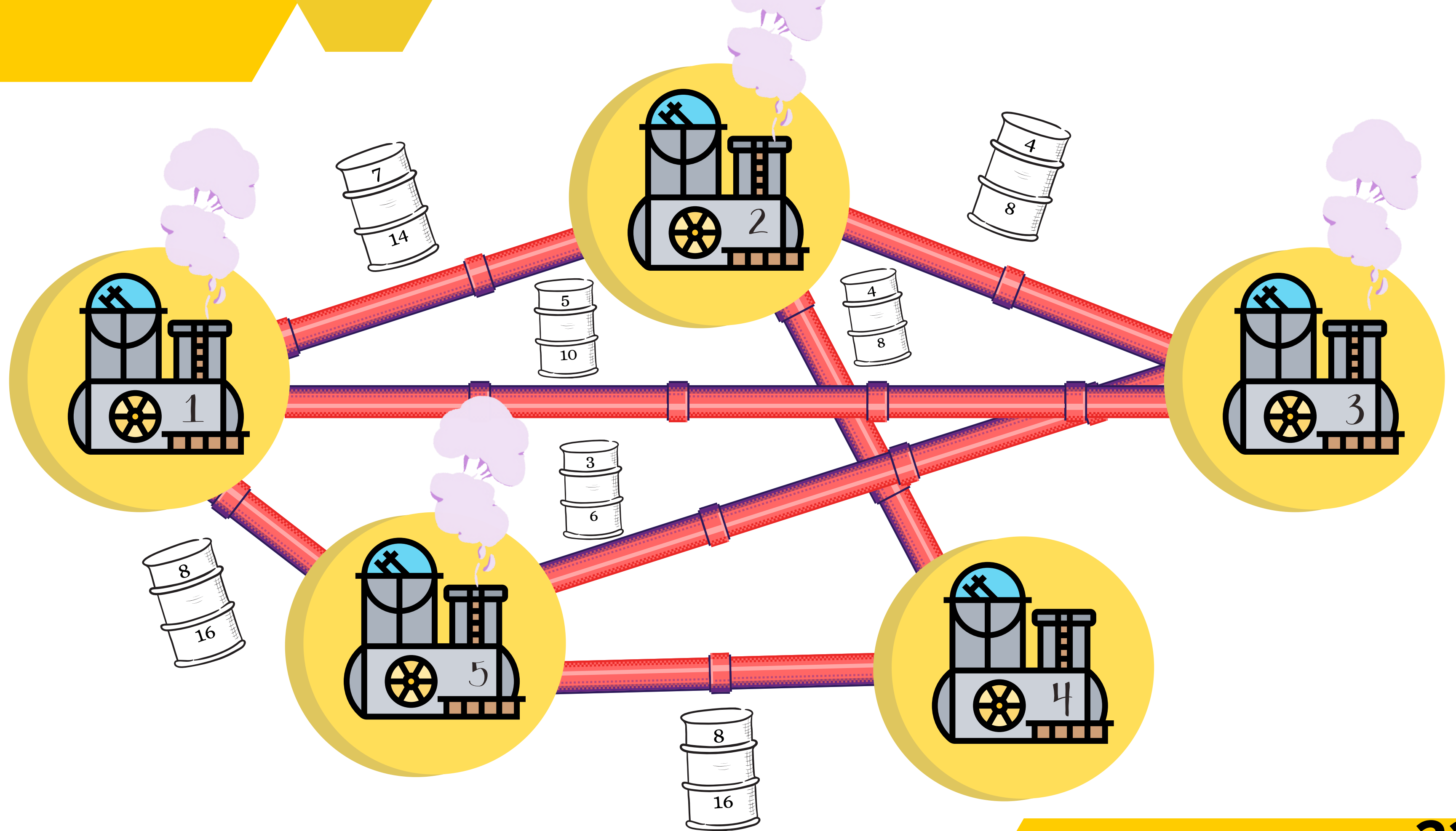












Les bibliothèques utilisées

- **Bibliothèque PuLP**

PuLP met à disposition tous les outils pour modéliser un problème et les résoudre en faisant appel à différents types de solveurs standards utilisant des algorithmes de résolution différents

- **Bibliothèque Pandas**

Pandas permet la manipulation et l'analyse des données. Elle propose en particulier des structures de données et des opérations de manipulation de tableaux numériques et de séries temporelles.

L'extraction des données

```
df = pd.read_excel(io: "SYS 2.xlsx", engine='openpyxl', header=None)
Problem = LpProblem(name: "Exemple1", LpMinimize)
# extraire la 1ere ligne du fichier excel
nbNoeuds, nbArcs, nbDemandes = df.iloc[0, :3].astype(int)
print(nbNoeuds, nbArcs, nbDemandes)

# Les données des liaisons sont les 16 lignes suivantes
colonnes_liaisons = ['i', 'j', 'nombre_liaisons', 'cout_liaison1', 'capacite_liaison1', 'cout_liaison2',
                    'capacite_liaison2']
donnees_liaisons = df.iloc[1:nbArcs + 1, :7]
donnees_liaisons.columns = colonnes_liaisons

# Les données des demandes suivent après les données des liaisons
colonnes_demandes = ['source', 'destination', 'quantite_demande']
donnees_demandes = df.iloc[nbArcs + 1:, :3]
```

Variables de décision

```
#variables decision
X = LpVariable.dicts("Connexion", [(i, j, k) for (i, j, k) in cap.keys()], cat='Binary')

#variable to define the flow of petrol in the zone ij for the demand sd
F = LpVariable.dicts("Flux",
                    [(i, j, k, s, d) for (i, j, k) in cap.keys() for (s, d) in D.keys()],
                    lowBound=0,
                    cat='Continuous')
```

Fonction Objectif

```
Problem += pulp.lpSum([C[(i, j, k)] * X[(i, j, k)] for (i, j, k) in liason])
```

Les sous contraintes

sur un arc au plus un type de connexion établie

```
# contrainte1: Au Plus un Type de Connexion entre 2 zones
for i in range(1, nbNoeuds + 1):
    for j in range(1, nbNoeuds + 1):
        Problem += pulp.lpSum(
            [X[(i, j, k)] for k in range(1, 3) if (i, j, k) in liason]) <= 1, f"Single_Connection_{i}_{j}"
```

Les sous contraintes

La demande est égale à la quantité de flux circulant entre source et destination qui lui correspond

```
# contrainte2:
for (s, d) in D.keys():
    for n in range(1, nbNoeuds + 1): # Assurez-vous que zones contient tous les nœuds (sommets) de votre réseau
        # Calculer le flux sortant de n pour la demande (s, d)
        flux_sortant = lpSum(F[i, j, k, s, d] for (i, j, k) in cap.keys() if i == n)
        # Calculer le flux entrant dans n pour la demande (s, d)
        flux_entrant = lpSum(F[i, j, k, s, d] for (i, j, k) in cap.keys() if j == n)

        # Si le sommet n est la source de la demande
        if n == s:
            Problem += (flux_sortant - flux_entrant == D[(s, d)], f"flux_net_{s}_{d}_source")
        # Si le sommet n est la destination de la demande
        elif n == d:
            Problem += (flux_sortant - flux_entrant == - D[(s, d)], f"flux_net_{s}_{d}_destination")
        # Si le sommet n n'est ni la source ni la destination
        else:
            Problem += (flux_sortant - flux_entrant == 0, f"flux_net_{s}_{d}_intermediate_{n}")
```


Les sous contraintes

La somme des flux circulant dans une pipeline est inférieure à sa capacité

```
#contrainte3:
# Ajouter la contrainte sur chaque liaison pour chaque type de liaison
for (i, j, k) in cap.keys():
    # Calcul de la somme des flux pour toutes les demandes, dans les deux sens de la liaison
    flux_total = lpSum(F[(i, j, k, s, d)] for (s, d) in D.keys() if (i, j, k, s, d) in F) + \
    lpSum(F[(j, i, k, s, d)] for (s, d) in D.keys() if (j, i, k, s, d) in F)

    # S'assurer que la somme des flux ne dépasse pas la capacité de la liaison,
    # multipliée par la variable binaire indiquant si la liaison est activée
    Problem += flux_total <= cap[(i, j, 1)] * X[(i, j, 1)] + cap[(i, j, 2)] * X[(i, j, 2)] ,
    f"Capacite_flux_total_{i}_{j}_{k}"

# Assurer que toutes les demandes sont satisfaites
for (s, d), quantite_demandee in D.items():
    # Calculer le flux total acheminé pour la demande de s à d sur toutes les liaisons et pour tous les types
    flux_total_pour_demande = lpSum(F[(i, j, k, s, d)]
    for i, j, k in cap.keys()
    if (i, j, k, s, d) in F)

    # Ajouter la contrainte que le flux total pour cette demande doit être au moins égal à la quantité demandée
    Problem += flux_total_pour_demande >= quantite_demandee, f"Satisfaction_demande_{s}_{d}"
```

La solution optimale

```
Objective value:          39.000000000
Enumerated nodes:         6
Total iterations:         704
Time (CPU seconds):       0.20
Time (Wallclock seconds): 0.20

Option for printingOptions changed from normal to all
Total time (CPU seconds):  0.21   (Wallclock seconds):  0.21
```

Le coût optimal :

39.0

Utilisation du modèle pour une autre DATA

En changeant les données , le modèle reste valide

```
Objective value:                24.000000000
Enumerated nodes:               3026
Total iterations:               429746
Time (CPU seconds):             183.29
Time (Wallclock seconds):       183.29

Option for printingOptions changed from normal to all
Total time (CPU seconds):       183.36   (Wallclock seconds):       183.36
```

Le coût optimal :

24.0



Heuristiques

MÉTHODE HEURISTIQUE

La méthode heuristique est une approche de résolution de problèmes qui utilise des règles pratiques tout en respectant les contraintes pour arriver à une solution qui s'approche de la solution exacte.



La première méthode

Cette méthode consiste à installer des pipelines de capacité maximale sur chaque liaison entre les sites de pétrole brut et les raffineries

```
import pulp
from pulp import*
import pandas as pd
df = pd.read_excel("SYS.xlsx", engine='openpyxl')
Problem=LpProblem("Exemple1", LpMinimize)
#extraire la 1ere ligne du fichier excel
nbNoeuds, nbArcs, nbDemandes = df.iloc[0, :3].astype(int)
# Les données des liaisons sont les 16 lignes suivantes
colonnes_liaisons = ['i', 'j', 'nombre_liaisons', 'cout_liaison1', 'capacite_liaison1', 'cout_liaison2', 'capacite_liaison2']
donnees_liaisons = df.iloc[0:16, :7]
donnees_liaisons.columns = colonnes_liaisons

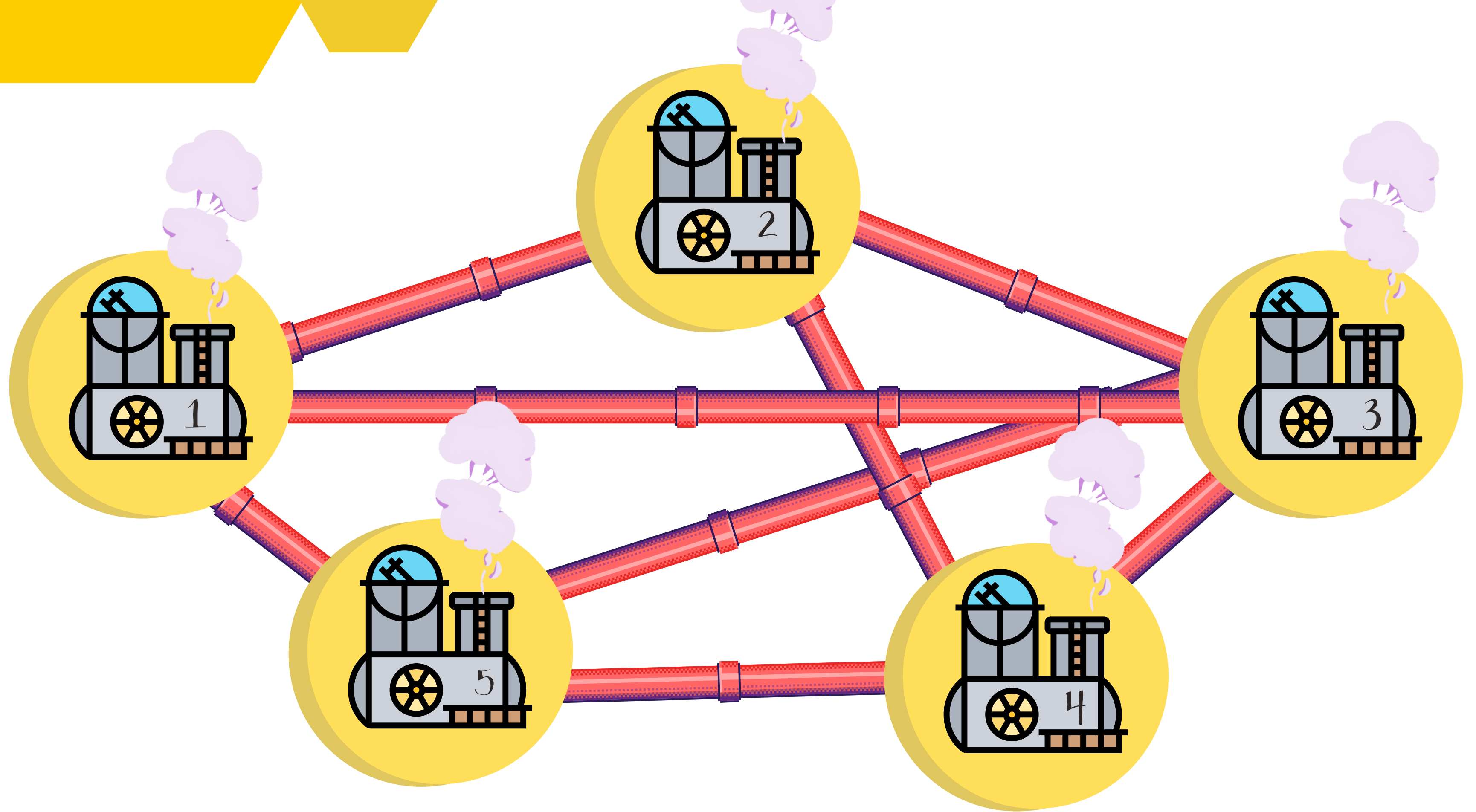
# Les données des demandes suivent après les données des liaisons
colonnes_demandes = ['source', 'destination', 'quantite_demande']
donnees_demandes = df.iloc[16:, :3]
donnees_demandes.columns = colonnes_demandes
liaisons = []
```

```
for index, row in donnees_liaisons.iterrows():
    capacite_maximale = max(row['capacite_liaison1'], row['capacite_liaison2'])
    if row['capacite_liaison1'] >= row['capacite_liaison2']:
        cout = row['cout_liaison1']
    else:
        cout = row['cout_liaison2']
    liaisons.append((capacite_maximale, cout))
print("voici le resultat")
print(liaisons)
print("voici le resultat final")
somme_totale_couts = sum(cout for _, cout in liaisons)

print("Somme totale des coûts pour toutes les liaisons :", somme_totale_couts/2)
```

La Somme totale des coûts pour toutes les liaisons :

52.0



La deuxième méthode

```
import pulp
from pulp import*
import numpy
import pandas as pd
df = pd.read_excel("SYS.xlsx", engine='openpyxl')
Problem=LpProblem("Exemple1", LpMinimize)
#extraire la 1ere ligne du fichier excel
nbNoeuds, nbArcs, nbDemandes = df.iloc[0, :3].astype(int)
# Les données des liaisons sont les 16 lignes suivantes
colonnes_liaisons = ['i', 'j', 'nombre_liaisons', 'cout_liaison1', 'capacite_liaison1','cout_liaison2','capacite_liaison2']
donnees_liaisons = df.iloc[0:16, :7]
donnees_liaisons.columns = colonnes_liaisons

# Les données des demandes suivent après les données des liaisons
colonnes_demandes = ['source', 'destination', 'quantite_demande']
donnees_demandes = df.iloc[16:, :3]
donnees_demandes.columns = colonnes_demandes
#define the decision variables
zones = range(1, 6) # Génère des indices de zone de 1 à 6
types_liaison = range(1, 3) # Génère des indices de type de liaison de 1 à 2
#binary variable that takes 1 if we used a pipeline type k in the zone ij
```

```

#define the parameters of the problem
#cout en fonction de zone et de type C[(i,j,k)]
C={}
for _, ligne in donnees_liaisons.iterrows():
    L=[int(ligne["cout_liaison1"]),int(ligne["cout_liaison2"])]
    i= int(ligne["i"])
    j= int(ligne["j"])
    C[(i,j,1)]=L[0]
    C[(i,j,2)]=L[1]
#capacite en fonction de zone et de type
cap={}
for _, ligne in donnees_liaisons.iterrows():
    L=[int(ligne["capacite_liaison1"]),int(ligne["capacite_liaison2"])]
    i= int(ligne["i"])
    j= int(ligne["j"])
    cap[(i,j,1)]=L[0]
    cap[(i,j,2)]=L[1]
#quantité de demande en fonction du zone D[(i,j)]
D={}
for _, ligne in donnees_demands.iterrows():
    i, j = int(ligne["source"]), int(ligne["destination"])
    D[(i, j)] = ligne["quantite_demande"]

```

```

for cle, capacite in cap.items():
    print(f"Cle: {cle}, capacite: {capacite}")
for cle,cout in C.items():
    print(f"Cle: {cle}, cout: {cout}")
for cle, quantite in D.items():
    print(f"Cle: {cle}, quantite: {quantite}")
# Initialisation du coût total
coût_total = 0
# Demande pour les paires de zones spécifiées
demandes_specifiques = {
    (1, 4): D[1, 4],
    (4, 1): D[4, 1],
    (2, 5): D[2, 5],
    (5, 2): D[5, 2]
}

# Calcul de la demande maximale
demande_max_specifique = max(demandes_specifiques.values())

# Structure pour stocker le choix optimal de pipeline pour chaque paire de zones
coût_total = 0

# Structure pour stocker la demande totale pour chaque paire de zones
demande_totale = {}

# Calcul de la demande totale pour chaque paire de zones
for (i, j), quantite in D.items():
    if (i<j):
        demande_totale[(i, j)] = quantite+D[j,i]

```



```

# Structure pour stocker le choix optimal de pipeline pour chaque paire de zones
choix_pipeline = {}

for (i, j), quantite_demandee in demande_totale.items():
    # Ajuster pour inclure la demande maximale spécifique, si nécessaire
    quantite_demandee_ajustee = quantite_demandee + demande_max_specifique

    coût_minimal = float('inf')
    type_optimal = None

    for k in types_liaison:
        if (i, j, k) in cap and cap[i, j, k] >= quantite_demandee_ajustee:
            coût = C[i, j, k]
            if coût < coût_minimal:
                coût_minimal = coût
                type_optimal = k
        else:
            if (i,j,k) in cap:
                if C[i,j,1]<C[i,j,2]:
                    coût_minimal = C[i,j,2]
                    type_optimal = 2
                else:
                    coût_minimal = C[i,j,1]
                    type_optimal = 1

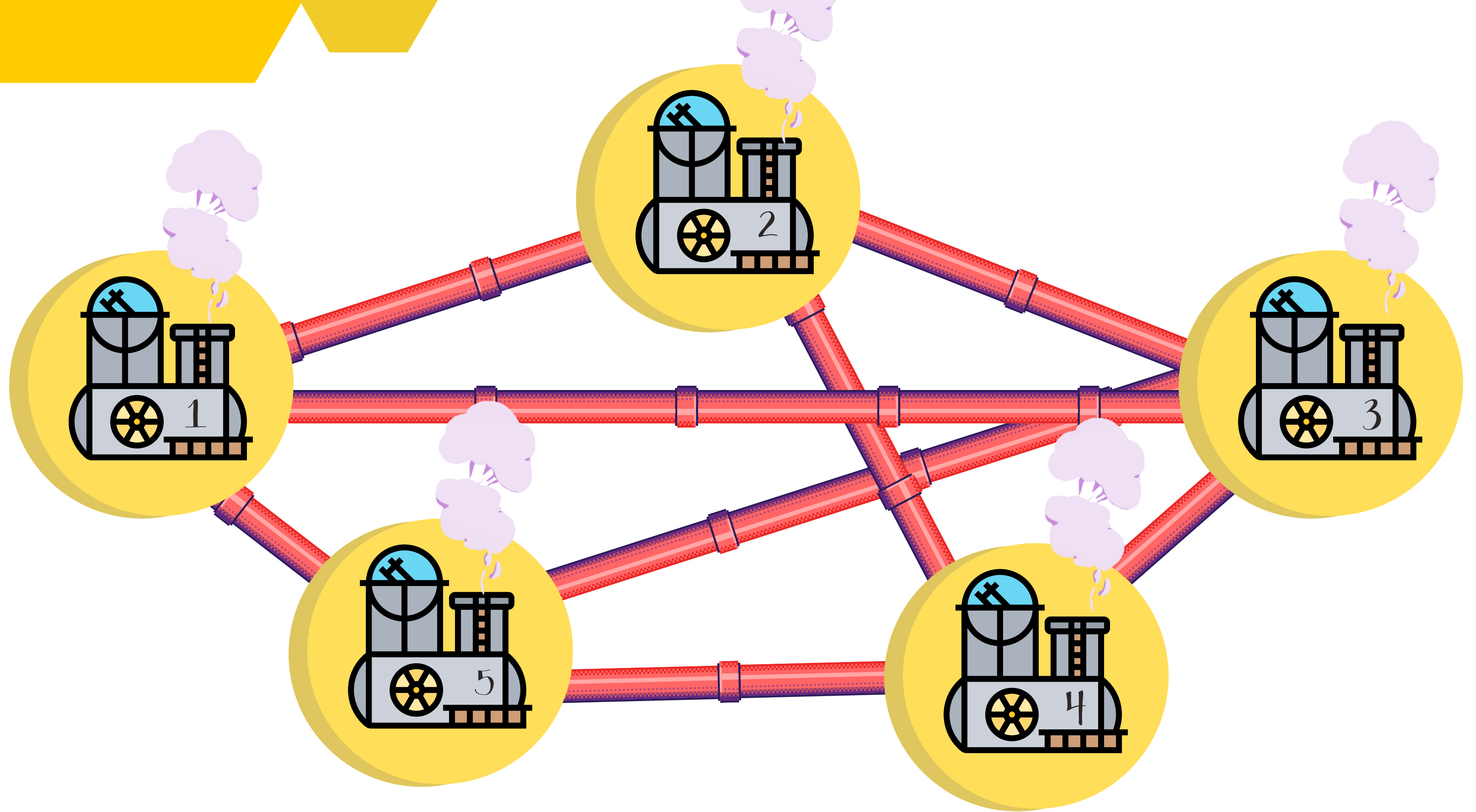
    if type_optimal is not None:
        # S'assurer de ne pas ajouter le coût pour la même liaison deux fois
        if (j, i) not in choix_pipeline:
            choix_pipeline[(i, j)] = (type_optimal, coût_minimal)
            coût_total += coût_minimal

```

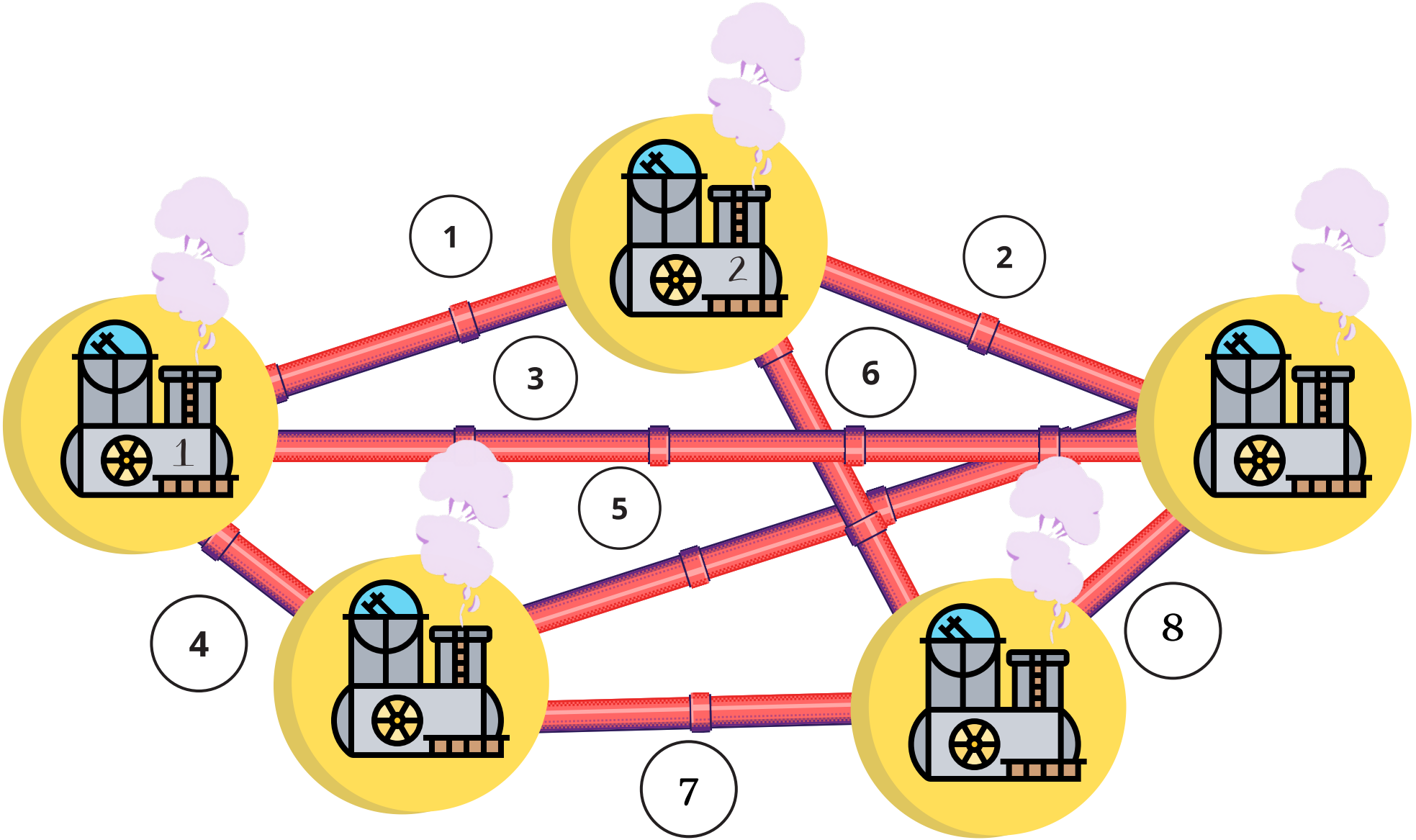
```
# Afficher le coût total d'installation
print(f"Le coût total d'installation est de {coût_total}.")
for cle,valeur in choix_pipeline.items():
    print(f"Cle: {cle}, type_cout: {valeur}")
```

La Somme totale des coûts pour toutes les liaisons :

46.0



N° Pipeline	Méthode 1	Méthode 2
1	7	7
2	6	6
3	6	6
4	8	6
5	3	3
6	4	4
7	11	8
8	7	6



Le GAP

La première méthode

$$\text{GAP1} = \frac{52-39}{39} * 100 = 33.3 \%$$

La deuxième méthode

$$\text{GAP2} = \frac{46-39}{39} * 100 = 17.9 \%$$

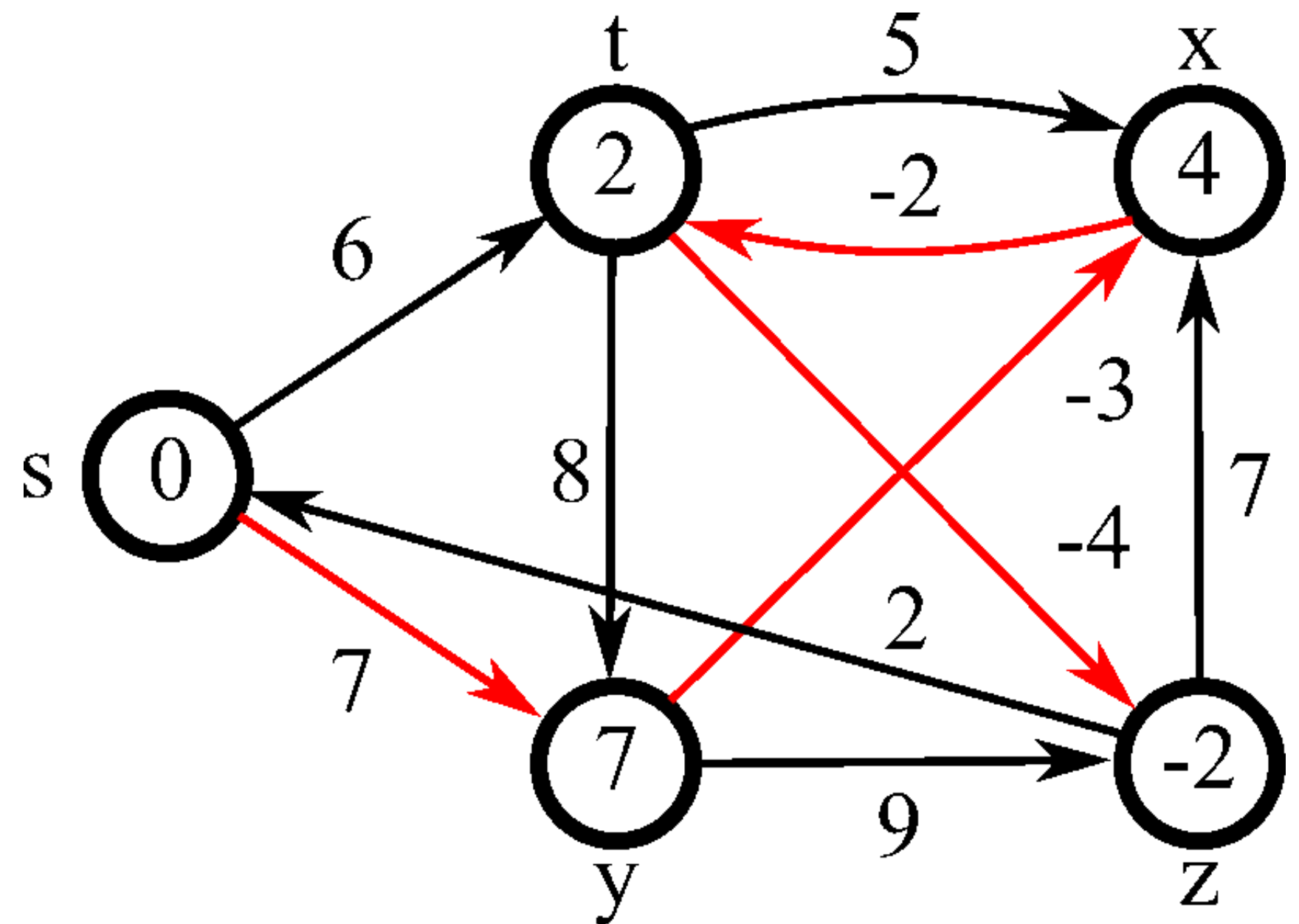
AUTRES METHODES DE RÉOLUTION

- **Algorithme de Bellman-Ford**
- **Algorithme de Ford-Fulkerson**
- **Algorithme de Busacker-Gowen**

Algorithme de Bellman-Ford

L'algorithme de Bellman-Ford est un **algorithme** qui calcule des plus courts chemins depuis un sommet source donné dans un **graphe orienté** pondéré.

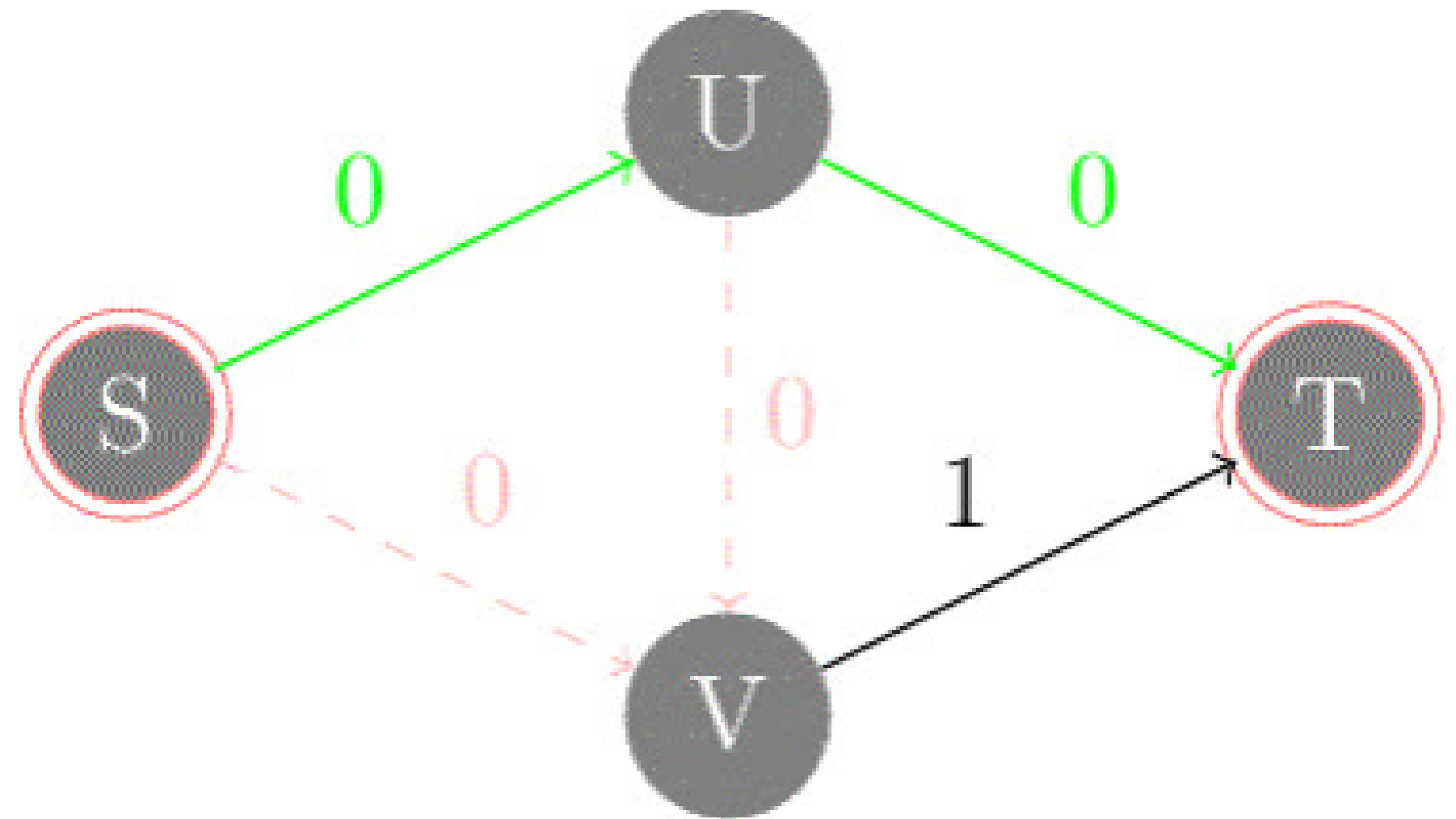
cet algorithme autorise la présence de certains arcs de poids négatif et permet de détecter l'existence d'un **circuit absorbant**.



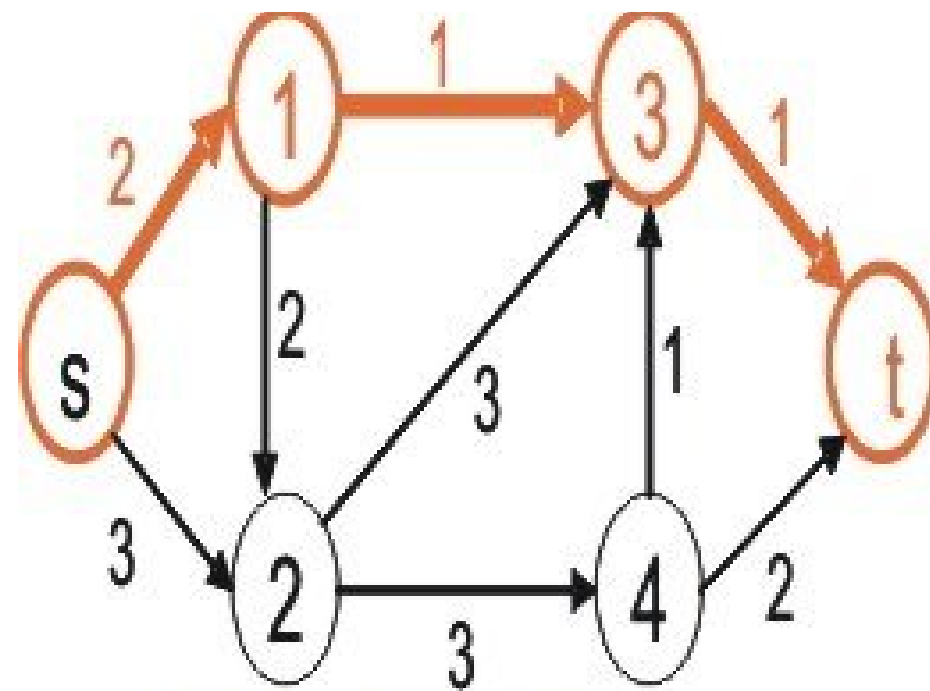
Algorithme de Ford-Fulkerson

En informatique, l'algorithme de Ford-Fulkerson est un algorithme pour le problème de flot maximum, un problème d'optimisation classique dans le domaine de **la recherche opérationnelle**.

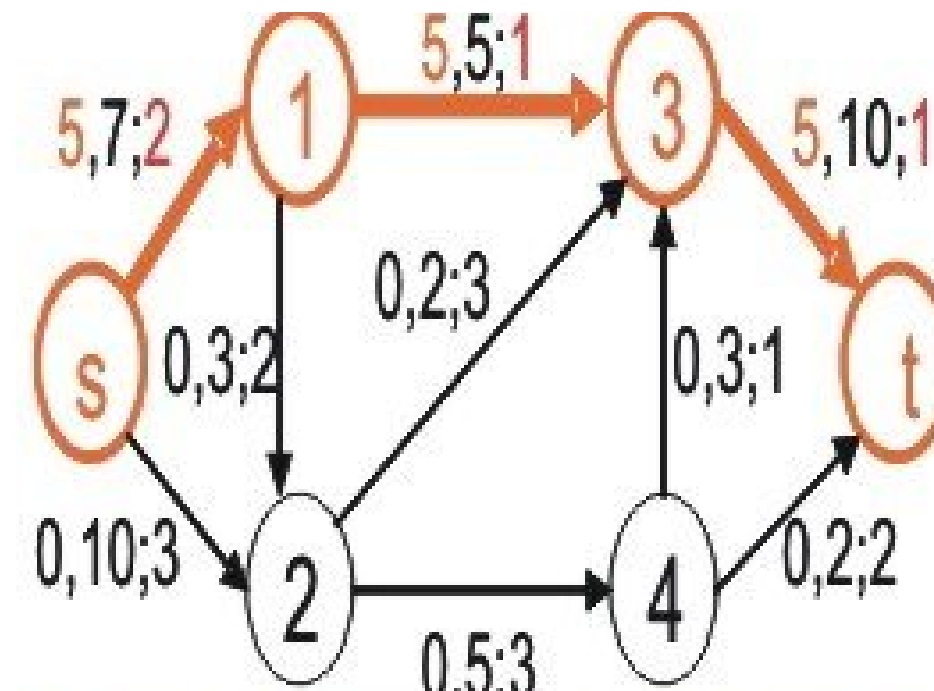
on utilise cet algorithme dans les applications sont multiples : problèmes informatiques, importations/exportations etc.



Algorithme de Busacker-Gowen



Plus court chemin



Flot ajouté, capacité maximale; coût

L'algorithme de Busacker et Gowen permet de résoudre le calcul V à **un coût minimal** dans un graphe $G=(X;U)$ où à chaque arc u de U est associé une capacité maximale C_u et un coût u .

Cet algorithme consiste à construire le graphe d'écart et ensuite à rechercher le chemin de coût minimal entre le nœud source s et le nœud de destination t .

Merci pour votre attention !